TABLE OF CONTENTS

1. PROJECT OVERVIEW

What is FreightFlow PRO?

A desktop application that replaces 10+ Excel sheets for freight forwarding companies.

The Problem: You type the same data (clients, containers, rates) into multiple Excel sheets. This causes typos, wastes time, and looks unprofessional.

The Solution: One system where you enter data ONCE. Everything links to a Job Number. Print professional documents with one click.

The Result: A job that takes 45 minutes in Excel takes 5 minutes in FreightFlow PRO.

Core Principle

Job_Number is the master key. Everything (rates, containers, documents, taxes) links to one Job_Number.

2. WHAT WE'RE BUILDING

Features (Complete List)

Phase 1: Foundation (Days 1-2)

✅ Database with 10 tables

✅ Sample data

✅ Helper functions

✅ REST API (all CRUD operations)

Phase 2: User Interface (Days 3-5)
✅ Dashboard (view all jobs)

✅ Create/Edit Job form

✅ Job Detail view

✅ Master Data management

Phase 3: Documents (Days 5-6)
✅ Print Bill of Lading (PDF)

✅ Print Invoice (PDF)

✅ Print Booking Note (PDF)

✅ Print CRO Request (PDF)

Phase 4: Reports (Day 7)
✅ GPSHT Report (freight tracking)

✅ JOBGP Report (profit analysis)

✅ Excel Export for both

Phase 5: Desktop Packaging (Days 8-9)
✅ Electron wrapper

✅ .exe installer

✅ Professional desktop app

What We're NOT Building

Feature	Why Not
Auto-forwarding	You want manual control
WhatsApp/Fax	Not needed
Mobile app	Keep it simple
User roles	Single user for now
Complex analytics	Excel export is enough
Multi-office	Not needed yet
Cloud hosting	Data stays local

3. TECHNOLOGY STACK
Complete Tech Stack

Component	Technology	Why

Database	SQLite	Single file, no setup, no server
Backend	Node.js + Express	Fast, simple, JavaScript everywhere
Frontend	HTML + CSS + JavaScript	No frameworks, lightweight
Desktop	Electron	Turns web app into desktop .exe
PDF	jsPDF	Generate PDFs locally
Excel	SheetJS (xlsx)	Export reports to Excel
Packaging	electron-builder	Creates installer .exe

4. COMPLETE FILE STRUCTURE

What Your Project Will Look Like

text

```
freightflow/
│
├── 📁 data/
│   └── freightflow.sqlite        ← YOUR DATABASE (created automatically)
│
├── 📁 public/                    ← Frontend files (Phase 2-7)
│   ├── index.html                ← Dashboard
│   ├── new-job.html              ← Create/Edit Job
│   ├── job-detail.html           ← Job Detail View
│   ├── master-data.html          ← Master Data Management
│   ├── reports-gpsht.html        ← GPSHT Report
│   ├── reports-jobgp.html        ← JOBGP Report
│   ├── style.css                 ← All styling
│   └── app.js                    ← Frontend logic
│
├── 📁 routes/                    ← API Routes (Day 2)
│   ├── jobs.js
│   ├── clients.js
│   ├── ports.js
│   ├── commodities.js
│   ├── containerTypes.js
│   └── reports.js
│
├── 📁 pdf/                       ← PDF Templates (Day 5-6)
│   ├── blTemplate.js
│   ├── invoiceTemplate.js
│   ├── bookingTemplate.js
│   └── croTemplate.js
│
├── 📁 exports/                   ← Exported Excel files (created automatically)
│
├── 📁 dist/                      ← .exe installer (created by electron-builder)
│
├── 📄 package.json               ← Dependencies + scripts
```

```
├── 📄 database.js          ← Database + helper functions (Day 1)
├── 📄 server.js            ← Express server (Day 2)
├── 📄 main.js              ← Electron entry point (Day 8)
├── 📄 test.js              ← Test script (Day 1)
├── 📄 .env                 ← Configuration
└── 📄 README.md                ← Instructions
```

5. ALL FILES (COMPLETE CODE)

📄 package.json

json

```
{
  "name": "freightflow-pro",
  "version": "1.0.0",
  "description": "Freight Forwarding Management System",
  "main": "main.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "node test.js",
    "db:init": "node database.js",
    "electron": "electron .",
    "build": "electron-builder",
    "build:win": "electron-builder --win --x64"
  },
  "author": "",
  "license": "MIT",
  "dependencies": {
    "express": "^4.18.2",
    "sqlite3": "^5.1.6",
    "cors": "^2.8.5",
    "dotenv": "^16.0.3",
    "body-parser": "^1.20.2",
    "jspdf": "^2.5.1",
    "jspdf-autotable": "^3.5.31",
    "xlsx": "^0.18.5",
    "electron": "^27.0.0",
    "electron-builder": "^24.6.4"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  },
  "build": {
    "appId": "com.freightflow.pro",
    "productName": "FreightFlow PRO",
    "directories": {
```

```
    "output": "dist"
  },
  "files": [
    "public/**/*",
    "routes/**/*",
    "pdf/**/*",
    "database.js",
    "server.js",
    "main.js",
    "package.json",
    ".env"
  ],
  "win": {
    "target": "nsis",
    "icon": "icon.ico"
  },
  "nsis": {
    "oneClick": false,
    "allowToChangeInstallationDirectory": true,
    "createDesktopShortcut": true,
    "createStartMenuShortcut": true
  }
 }
}
```

📄 .env

```
env
PORT=3000
DB_PATH=./data/freightflow.sqlite
EXCHANGE_RATE=280
```

📄 database.js

```
javascript
const sqlite3 = require('sqlite3').verbose();
const path = require('path');
const fs = require('fs');

// Ensure data directory exists
const dataDir = path.join(__dirname, 'data');
if (!fs.existsSync(dataDir)) {
  fs.mkdirSync(dataDir);
}

const dbPath = path.join(dataDir, 'freightflow.sqlite');
const db = new sqlite3.Database(dbPath);
```

```
// Helper to run queries with promises
function runQuery(query, params = []) {
    return new Promise((resolve, reject) => {
        db.run(query, params, function(err) {
            if (err) reject(err);
            else resolve({ id: this.lastID, changes: this.changes });
        });
    });
}

function getQuery(query, params = []) {
    return new Promise((resolve, reject) => {
        db.get(query, params, (err, row) => {
            if (err) reject(err);
            else resolve(row);
        });
    });
}

function allQuery(query, params = []) {
    return new Promise((resolve, reject) => {
        db.all(query, params, (err, rows) => {
            if (err) reject(err);
            else resolve(rows);
        });
    });
}

// ==================== INIT DATABASE ====================

async function initDatabase() {
    console.log('📦 Creating database tables...');

    // Master Data Tables
    await runQuery(`
        CREATE TABLE IF NOT EXISTS clients (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            type TEXT NOT NULL CHECK(type IN ('SHIPPER', 'CONSIGNEE', 'NOTIFY',
'VENDOR')),
            address TEXT,
            phone TEXT,
            email TEXT,
            contact_person TEXT,
```

```
        tax_id TEXT
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS ports (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        code TEXT,
        country TEXT
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS commodities (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        hs_code TEXT,
        description TEXT
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS container_types (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        code TEXT NOT NULL UNIQUE,
        description TEXT,
        weight_limit DECIMAL
    )
`);

// Transaction Tables
await runQuery(`
    CREATE TABLE IF NOT EXISTS jobs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        job_number TEXT UNIQUE NOT NULL,
        created_date DATE DEFAULT CURRENT_DATE,
        shipper_id INTEGER,
        consignee_id INTEGER,
        notify_1_id INTEGER,
        notify_2_id INTEGER,
        commodity_id INTEGER,
        pod_id INTEGER,
        bl_number TEXT,
```

```
        packages INTEGER,
        gross_weight DECIMAL,
        net_weight DECIMAL,
        marks TEXT,
        fin_number TEXT,
        etd DATE,
        eta DATE,
        status TEXT DEFAULT 'BOOKED' CHECK(status IN ('BOOKED', 'SAILED',
'DELIVERED', 'CLOSED')),
        notes TEXT,
        internal_notes TEXT,
        FOREIGN KEY (shipper_id) REFERENCES clients(id),
        FOREIGN KEY (consignee_id) REFERENCES clients(id),
        FOREIGN KEY (notify_1_id) REFERENCES clients(id),
        FOREIGN KEY (notify_2_id) REFERENCES clients(id),
        FOREIGN KEY (commodity_id) REFERENCES commodities(id),
        FOREIGN KEY (pod_id) REFERENCES ports(id)
      )
    `);

    await runQuery(`
      CREATE TABLE IF NOT EXISTS containers (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        job_id INTEGER NOT NULL,
        container_number TEXT,
        container_type_id INTEGER,
        seal_number TEXT,
        vessel TEXT,
        voyage TEXT,
        status TEXT DEFAULT 'EMPTY' CHECK(status IN ('EMPTY', 'FULL', 'INTRANSIT',
'DELIVERED')),
        pickup_date DATE,
        delivery_date DATE,
        FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE,
        FOREIGN KEY (container_type_id) REFERENCES container_types(id)
      )
    `);

    await runQuery(`
      CREATE TABLE IF NOT EXISTS rates (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        job_id INTEGER NOT NULL,
        rate_type TEXT NOT NULL CHECK(rate_type IN ('BUYING', 'SELLING')),
        charge_type TEXT NOT NULL,
```

```
      amount DECIMAL,
      currency TEXT DEFAULT 'USD' CHECK(currency IN ('USD', 'PKR')),
      vendor_id INTEGER,
      invoice_number TEXT,
      paid_status TEXT DEFAULT 'UNPAID' CHECK(paid_status IN ('UNPAID', 'PAID')),
      FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE,
      FOREIGN KEY (vendor_id) REFERENCES clients(id)
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS taxes (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      job_id INTEGER NOT NULL,
      tax_type TEXT NOT NULL CHECK(tax_type IN ('ZKT', 'KHRT')),
      percentage DECIMAL,
      base_amount DECIMAL,
      amount DECIMAL,
      paid_status TEXT DEFAULT 'UNPAID' CHECK(paid_status IN ('UNPAID', 'PAID')),
      paid_date DATE,
      paid_ref TEXT,
      FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS documents (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      job_id INTEGER NOT NULL,
      doc_type TEXT NOT NULL CHECK(doc_type IN ('BL', 'INVOICE', 'BOOKING', 'CRO')),
      doc_number TEXT,
      file_path TEXT,
      generated_date DATE,
      sent_date DATE,
      sent_to TEXT CHECK(sent_to IN ('CLIENT', 'CONSIGNEE', 'BANK')),
      sent_method TEXT CHECK(sent_method IN ('EMAIL', 'PRINT', 'COURIER')),
      FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
    )
`);

await runQuery(`
    CREATE TABLE IF NOT EXISTS bl_data (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      job_id INTEGER NOT NULL,
```

```
            bl_number TEXT,
            shipper_id INTEGER,
            consignee_id INTEGER,
            notify_1_id INTEGER,
            notify_2_id INTEGER,
            vessel TEXT,
            voyage TEXT,
            port_loading TEXT,
            port_discharge TEXT,
            port_delivery TEXT,
            marks TEXT,
            packages INTEGER,
            gross_weight DECIMAL,
            net_weight DECIMAL,
            commodity_desc TEXT,
            fin_number TEXT,
            freight_terms TEXT CHECK(freight_terms IN ('PREPAID', 'COLLECT')),
            free_days INTEGER,
            container_no TEXT,
            issued_date DATE,
            FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE,
            FOREIGN KEY (shipper_id) REFERENCES clients(id),
            FOREIGN KEY (consignee_id) REFERENCES clients(id),
            FOREIGN KEY (notify_1_id) REFERENCES clients(id),
            FOREIGN KEY (notify_2_id) REFERENCES clients(id)
        )
    `);

    console.log('✅ Database tables created');

    // Seed initial data
    await seedData();
}

// ==================== SEED DATA ====================

async function seedData() {
    console.log('🌱 Seeding sample data...');

    // Check if data already exists
    const existing = await getQuery('SELECT COUNT(*) as count FROM clients');
    if (existing.count > 0) {
        console.log('⚠️ Data already exists, skipping seed');
        return;
```

```
  }

  // Insert Clients
  const clients = [
    ['M.MUNIR AND SONS', 'SHIPPER', 'RAILWAY ROAD, RIYADH', '1234567',
'info@mmunir.com', 'Mr. Munir', 'TAX-001'],
    ['BANK ALFALAH', 'CONSIGNEE', 'BANK ROAD, LAHORE', '7654321',
'info@bankalfalah.com', 'Mr. Ali', 'TAX-002'],
    ['DIHA AL MARIFA TRADING EST', 'NOTIFY', 'P.O.BOX 31799, ALKHOBAR 31952',
'+966500189429', 'dm.ksa@yahoo.com', 'Mr. Ahmed', 'TAX-003'],
    ['DURRA SABAH TRADING COMPANY', 'NOTIFY', 'AL SULAE AREA, RIYADH',
'+966500189429', 'dm.ksa@yahoo.com', 'Mr. Khalid', 'TAX-004'],
    ['MEEZAN BANK LIMITED', 'CONSIGNEE', 'ZAHOOR ELAHI ROAD, LAHORE', '1122334',
'info@meezan.com', 'Mr. Hassan', 'TAX-005'],
    ['MEPCO CHEMICALS', 'SHIPPER', '16.5 KM SHEIKHU, JEDDAH', '5544332',
'info@mepco.com', 'Mr. Fahad', 'TAX-006'],
    ['ASIA ENTERPRISE', 'SHIPPER', 'CHAH BALANDAY, KARACHI', '6677889',
'info@asia.com', 'Mr. Imran', 'TAX-007'],
    ['KTM LEATHER', 'SHIPPER', 'MEHR MANZIL, LC QINGDAO', '9988776', 'info@ktm.com',
'Mr. Raza', 'TAX-008'],
    ['TAQ ENTP CARGO SERVICES PVT LTD', 'VENDOR', 'CARGO ROAD, KARACHI',
'4433221', 'info@taq.com', 'Mr. Tariq', 'TAX-009'],
    ['H&H ENTERPRISES', 'VENDOR', 'ENTERPRISE ROAD, LAHORE', '5566778',
'info@hh.com', 'Mr. Hamid', 'TAX-010']
  ];

  for (const c of clients) {
    await runQuery(
      'INSERT INTO clients (name, type, address, phone, email, contact_person, tax_id)
VALUES (?,?,?,?,?,?,?)',
      c
    );
  }

  // Insert Ports
  const ports = [
    ['JEDDAH', 'JED', 'SAUDI ARABIA'],
    ['DAMMAM', 'DAM', 'SAUDI ARABIA'],
    ['RIYADH', 'RUH', 'SAUDI ARABIA'],
    ['KARACHI', 'KHI', 'PAKISTAN'],
    ['QINGDAO', 'QIN', 'CHINA'],
    ['JEBELALI', 'JEA', 'UAE'],
    ['HAMAD', 'HAM', 'QATAR']
  ];
```

```
    for (const p of ports) {
        await runQuery('INSERT INTO ports (name, code, country) VALUES (?,?,?)', p);
    }

    // Insert Commodities
    const commodities = [
        ['TENT', '6306', 'Tents and camping equipment'],
        ['NIWAR', '6306', 'Canvas and ropes'],
        ['CANVAS', '6306', 'Canvas rolls'],
        ['LEATHER', '4107', 'Leather goods'],
        ['CHEMICALS', '2800', 'Industrial chemicals']
    ];

    for (const c of commodities) {
        await runQuery('INSERT INTO commodities (name, hs_code, description) VALUES
(?,?,?)', c);
    }

    // Insert Container Types
    const containerTypes = [
        ['1x20', '20-foot container', 28000],
        ['1x40', '40-foot container', 30000],
        ['1x40HC', '40-foot high cube', 30000],
        ['20RFR', '20-foot reefer', 27000],
        ['40RFR', '40-foot reefer', 29000]
    ];

    for (const ct of containerTypes) {
        await runQuery('INSERT INTO container_types (code, description, weight_limit) VALUES
(?,?,?)', ct);
    }

    // Insert Sample Job (EUMEX-2026-001)
    const jobResult = await runQuery(`
        INSERT INTO jobs (
            job_number, created_date, shipper_id, consignee_id, notify_1_id, notify_2_id,
            commodity_id, pod_id, bl_number, packages, gross_weight, net_weight,
            marks, fin_number, etd, eta, status
        ) VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
    `, [
        'EUMEX-2026-001', '2026-01-20',
        1, 2, 3, 4,
        1, 1,
```

```
        '078G100088', 193, 15450, 15350,
        '193', 'MBL-EXP-861186-12122025',
        '2026-01-25', '2026-02-10',
        'BOOKED'
    ]);

    const jobId = jobResult.id;

    // Insert Container
    await runQuery(`
        INSERT INTO containers (job_id, container_number, container_type_id, vessel, voyage,
status)
        VALUES (?,?,?,?,?,?)
    `, [jobId, 'WHUS667790', 2, 'CELSIUS EMMEN 20', '20', 'FULL']);

    // Insert Rates - Buying
    const buyingRates = [
        ['FREIGHT', 1110, 'USD'],
        ['PLACEMENT', 100, 'USD'],
        ['LIFT ON', 25, 'USD'],
        ['SEAL', 7, 'USD'],
        ['BL CHARGES', 85, 'USD'],
        ['CRO', 3, 'USD'],
        ['SURRENDER/TLX', 50, 'USD'],
        ['LIFT OFF', 2700, 'USD']
    ];

    for (const r of buyingRates) {
        await runQuery(
            'INSERT INTO rates (job_id, rate_type, charge_type, amount, currency, vendor_id)
VALUES (?,?,?,?,?,?)',
            [jobId, 'BUYING', r[0], r[1], r[2], 9]
        );
    }

    // Insert Rates - Selling
    const sellingRates = [
        ['FREIGHT', 1500, 'USD'],
        ['PLACEMENT', 150, 'USD'],
        ['LIFT ON', 40, 'USD'],
        ['SEAL', 10, 'USD'],
        ['BL CHARGES', 120, 'USD'],
        ['CRO', 5, 'USD']
    ];
```

```
    for (const r of sellingRates) {
        await runQuery(
            'INSERT INTO rates (job_id, rate_type, charge_type, amount, currency) VALUES
(?,?,?,?,?)',
            [jobId, 'SELLING', r[0], r[1], r[2]]
        );
    }

    console.log('✅ Sample data seeded');
    console.log(`📦 Sample job created: EUMEX-2026-001 (ID: ${jobId})`);
}

// ==================== HELPER FUNCTIONS ====================

// Get all records from a table
async function getAll(tableName) {
    return await allQuery(`SELECT * FROM ${tableName}`);
}

// Get one record by ID
async function getById(tableName, id) {
    return await getQuery(`SELECT * FROM ${tableName} WHERE id = ?`, [id]);
}

// Create a new record
async function create(tableName, data) {
    const keys = Object.keys(data);
    const placeholders = keys.map(() => '?').join(',');
    const query = `INSERT INTO ${tableName} (${keys.join(',')}) VALUES (${placeholders})`;
    const values = keys.map(key => data[key]);
    return await runQuery(query, values);
}

// Update a record
async function update(tableName, id, data) {
    const keys = Object.keys(data);
    const setClause = keys.map(key => `${key} = ?`).join(',');
    const query = `UPDATE ${tableName} SET ${setClause} WHERE id = ?`;
    const values = [...keys.map(key => data[key]), id];
    return await runQuery(query, values);
}

// Delete a record
```

```
async function deleteRecord(tableName, id) {
    return await runQuery(`DELETE FROM ${tableName} WHERE id = ?`, [id]);
}

// Get all jobs with details (for dashboard)
async function getJobsWithDetails() {
    return await allQuery(`
        SELECT
            j.id, j.job_number, j.created_date, j.bl_number,
            j.packages, j.gross_weight, j.net_weight, j.marks,
            j.etd, j.eta, j.status,
            s.name as shipper,
            c.name as consignee,
            co.name as commodity,
            p.name as pod,
            (SELECT SUM(amount) FROM rates WHERE job_id = j.id AND rate_type = 'SELLING')
as total_selling,
            (SELECT SUM(amount) FROM rates WHERE job_id = j.id AND rate_type = 'BUYING')
as total_buying
        FROM jobs j
        LEFT JOIN clients s ON j.shipper_id = s.id
        LEFT JOIN clients c ON j.consignee_id = c.id
        LEFT JOIN commodities co ON j.commodity_id = co.id
        LEFT JOIN ports p ON j.pod_id = p.id
        ORDER BY j.id DESC
    `);
}

// Get one job with all related data
async function getJobById(id) {
    const job = await getQuery(`
        SELECT
            j.*,
            s.name as shipper_name, s.address as shipper_address,
            c.name as consignee_name, c.address as consignee_address,
            n1.name as notify_1_name, n1.address as notify_1_address,
            n2.name as notify_2_name, n2.address as notify_2_address,
            co.name as commodity_name,
            p.name as pod_name, p.code as pod_code
        FROM jobs j
        LEFT JOIN clients s ON j.shipper_id = s.id
        LEFT JOIN clients c ON j.consignee_id = c.id
        LEFT JOIN clients n1 ON j.notify_1_id = n1.id
        LEFT JOIN clients n2 ON j.notify_2_id = n2.id
```

```
    LEFT JOIN commodities co ON j.commodity_id = co.id
    LEFT JOIN ports p ON j.pod_id = p.id
    WHERE j.id = ?
`, [id]);

if (!job) return null;

// Get containers
const containers = await allQuery(`
    SELECT c.*, ct.code as container_type_code
    FROM containers c
    LEFT JOIN container_types ct ON c.container_type_id = ct.id
    WHERE c.job_id = ?
`, [id]);

// Get rates
const rates = await allQuery(`
    SELECT r.*, cl.name as vendor_name
    FROM rates r
    LEFT JOIN clients cl ON r.vendor_id = cl.id
    WHERE r.job_id = ?
`, [id]);

// Get taxes
const taxes = await allQuery(`
    SELECT * FROM taxes WHERE job_id = ?
`, [id]);

// Calculate profit
const profit = await calculateProfit(id);

// Get BL data
const blData = await getQuery(`
    SELECT * FROM bl_data WHERE job_id = ?
`, [id]);

return {
    ...job,
    containers,
    rates,
    taxes,
    profit,
    bl_data: blData
};
```

```
}

// Get next job number
async function getNextJobNumber() {
    const result = await getQuery(`
        SELECT job_number FROM jobs
        ORDER BY id DESC LIMIT 1
    `);

    if (!result) {
        const year = new Date().getFullYear();
        return `EUMEX-${year}-001`;
    }

    const parts = result.job_number.split('-');
    const year = new Date().getFullYear();
    const currentYear = parseInt(parts[1]);
    let sequence = parseInt(parts[2]);

    if (currentYear < year) {
        return `EUMEX-${year}-001`;
    }

    sequence = sequence + 1;
    const paddedSeq = String(sequence).padStart(3, '0');
    return `EUMEX-${year}-${paddedSeq}`;
}

// Calculate profit for a job
async function calculateProfit(jobId) {
    const rates = await allQuery(`
        SELECT rate_type, SUM(amount) as total
        FROM rates
        WHERE job_id = ?
        GROUP BY rate_type
    `, [jobId]);

    let totalBuying = 0;
    let totalSelling = 0;

    for (const r of rates) {
        if (r.rate_type === 'BUYING') totalBuying = r.total || 0;
        if (r.rate_type === 'SELLING') totalSelling = r.total || 0;
    }
```

```
    const profitUSD = totalSelling - totalBuying;
    const exchangeRate = parseFloat(process.env.EXCHANGE_RATE || 280);
    const profitPKR = profitUSD * exchangeRate;

    return {
        total_buying: totalBuying,
        total_selling: totalSelling,
        profit_usd: profitUSD,
        profit_pkr: profitPKR,
        exchange_rate: exchangeRate
    };
}

// Generate taxes for a job
async function generateTaxes(jobId) {
    const profit = await calculateProfit(jobId);
    const profitPKR = profit.profit_pkr;

    // Only calculate taxes if profit is positive
    let zktAmount = 0;
    let khtrAmount = 0;

    if (profitPKR > 0) {
        zktAmount = profitPKR * 0.025; // 2.5%
        khtrAmount = profitPKR * 0.075; // 7.5%
    }

    // Delete existing taxes
    await runQuery('DELETE FROM taxes WHERE job_id = ?', [jobId]);

    // Insert ZKT
    await runQuery(`
        INSERT INTO taxes (job_id, tax_type, percentage, base_amount, amount)
        VALUES (?, 'ZKT', 2.5, ?, ?)
    `, [jobId, profitPKR, zktAmount]);

    // Insert KHRT
    await runQuery(`
        INSERT INTO taxes (job_id, tax_type, percentage, base_amount, amount)
        VALUES (?, 'KHRT', 7.5, ?, ?)
    `, [jobId, profitPKR, khtrAmount]);

    return {
```

```
        zkt: zktAmount,
        khtr: khtrAmount,
        total: zktAmount + khtrAmount
    };
}

// Get GPSHT Report
async function getGPSHTReport() {
    return await allQuery(`
        SELECT
            j.job_number,
            s.name as shipper,
            c.container_number,
            c.vessel,
            c.voyage,
            j.etd,
            j.eta,
            j.status,
            j.bl_number,
            p.name as pod
        FROM jobs j
        LEFT JOIN clients s ON j.shipper_id = s.id
        LEFT JOIN containers c ON j.id = c.job_id
        LEFT JOIN ports p ON j.pod_id = p.id
        ORDER BY j.etd DESC
    `);
}

// Get JOBGP Report
async function getJOBGPReport() {
    const jobs = await getJobsWithDetails();
    const results = [];

    for (const job of jobs) {
        const profit = await calculateProfit(job.id);
        const taxes = await allQuery(`
            SELECT tax_type, amount FROM taxes WHERE job_id = ?
        `, [job.id]);

        let zkt = 0, khtr = 0;
        for (const t of taxes) {
            if (t.tax_type === 'ZKT') zkt = t.amount || 0;
            if (t.tax_type === 'KHRT') khtr = t.amount || 0;
        }
```

```
        // Calculate net GP (profit after taxes)
        const profitPKR = profit.profit_pkr || 0;
        const netGP = profitPKR - zkt - khtr;

        results.push({
            job_number: job.job_number,
            shipper: job.shipper,
            freight_received: profit.total_selling || 0,
            freight_paid: profit.total_buying || 0,
            profit_usd: profit.profit_usd || 0,
            profit_pkr: profitPKR,
            zkt: zkt,
            khtr: khtr,
            net_gp: netGP,
            status: job.status
        });
    }

    return results;
}

// ==================== EXPORTS ====================

module.exports = {
    db,
    initDatabase,
    getAll,
    getById,
    create,
    update,
    deleteRecord,
    getJobsWithDetails,
    getJobById,
    getNextJobNumber,
    calculateProfit,
    generateTaxes,
    getGPSHTReport,
    getJOBGPReport,
    runQuery,
    getQuery,
    allQuery
};
```

📄 test.js

```
javascript
const db = require('./database.js');

async function runTests() {
    console.log('🧪 Running tests...\n');

    let passed = 0;
    let failed = 0;

    // Test 1: Database exists
    try {
        const tables = await db.allQuery("SELECT name FROM sqlite_master WHERE
type='table'");
        const tableNames = tables.map(t => t.name);
        const expectedTables = ['clients', 'ports', 'commodities', 'container_types', 'jobs',
'containers', 'rates', 'taxes', 'documents', 'bl_data'];

        const allExist = expectedTables.every(t => tableNames.includes(t));
        if (allExist) {
            console.log('✅ All 10 tables exist');
            passed++;
        } else {
            console.log('❌ Missing tables:', expectedTables.filter(t => !tableNames.includes(t)));
            failed++;
        }
    } catch (e) {
        console.log('❌ Test 1 failed:', e.message);
        failed++;
    }

    // Test 2: Sample data exists
    try {
        const clients = await db.getAll('clients');
        if (clients.length >= 10) {
            console.log(`✅ ${clients.length} clients found`);
            passed++;
        } else {
            console.log(`❌ Expected 10+ clients, found ${clients.length}`);
            failed++;
        }
    } catch (e) {
        console.log('❌ Test 2 failed:', e.message);
        failed++;
    }
```

```
    // Test 3: Sample job exists
    try {
        const job = await db.getQuery("SELECT * FROM jobs WHERE job_number =
'EUMEX-2026-001'");
        if (job) {
            console.log(`✅ Sample job found: ${job.job_number}`);
            passed++;
        } else {
            console.log('❌ Sample job EUMEX-2026-001 not found');
            failed++;
        }
    } catch (e) {
        console.log('❌ Test 3 failed:', e.message);
        failed++;
    }

    // Test 4: Container linked to job
    try {
        const job = await db.getQuery("SELECT id FROM jobs WHERE job_number =
'EUMEX-2026-001'");
        if (job) {
            const containers = await db.allQuery("SELECT * FROM containers WHERE job_id = ?",
[job.id]);
            if (containers.length > 0) {
                console.log(`✅ ${containers.length} container(s) found for sample job`);
                passed++;
            } else {
                console.log('❌ No containers found for sample job');
                failed++;
            }
        }
    } catch (e) {
        console.log('❌ Test 4 failed:', e.message);
        failed++;
    }

    // Test 5: Rates exist
    try {
        const job = await db.getQuery("SELECT id FROM jobs WHERE job_number =
'EUMEX-2026-001'");
        if (job) {
            const rates = await db.allQuery("SELECT * FROM rates WHERE job_id = ?", [job.id]);
            if (rates.length >= 10) {
```

```
            console.log(`✅ ${rates.length} rates found for sample job`);
            passed++;
        } else {
            console.log(`❌ Expected 10+ rates, found ${rates.length}`);
            failed++;
        }
    }
} catch (e) {
    console.log('❌ Test 5 failed:', e.message);
    failed++;
}

// Test 6: Get next job number
try {
    const nextNum = await db.getNextJobNumber();
    console.log(`✅ Next job number: ${nextNum}`);
    passed++;
} catch (e) {
    console.log('❌ Test 6 failed:', e.message);
    failed++;
}

// Test 7: Calculate profit
try {
    const job = await db.getQuery("SELECT id FROM jobs WHERE job_number =
'EUMEX-2026-001'");
    if (job) {
        const profit = await db.calculateProfit(job.id);
        console.log(`✅ Profit calculated: USD ${profit.profit_usd}, PKR ${profit.profit_pkr}`);
        passed++;
    }
} catch (e) {
    console.log('❌ Test 7 failed:', e.message);
    failed++;
}

// Test 8: Generate taxes
try {
    const job = await db.getQuery("SELECT id FROM jobs WHERE job_number =
'EUMEX-2026-001'");
```